nixlab – Project documentation

As of: 2026-10-19

> This documentation describes a private repo. The domain and internal addresses are replaced with example values: the domain with `homelab.example` (RFC 2606), IPv4 per RFC 5737 (`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`), IPv6 per RFC 3849 (`2001:db8::/32`), the tailnet name with `example-tailnet.ts.net`. The real mesh range is a /22 out of RFC 1918. Modules, documentation and build tooling were written together with Claude (Anthropic). License: MIT.

Overview

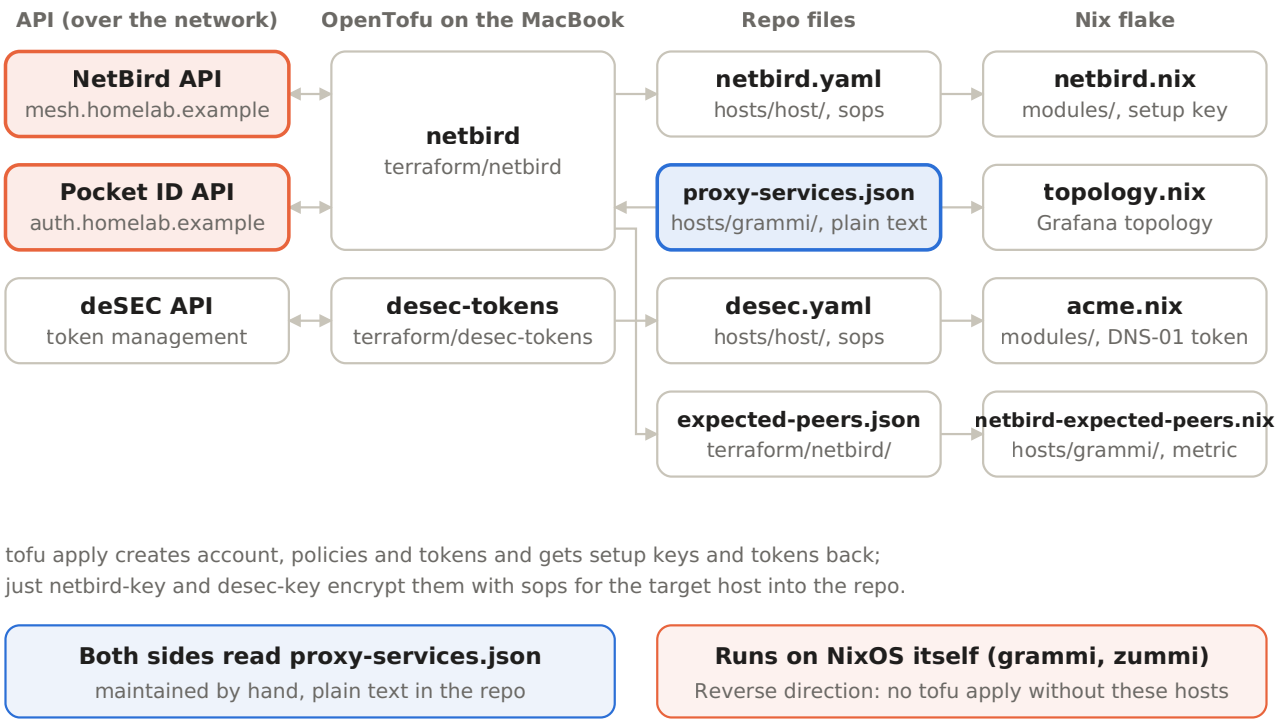
nixlab is the flake monorepo for the **homelab.example** homelab: five NixOS hosts and a Mac mini running nix-darwin, plus three OpenTofu modules for the NetBird account, the deSEC API tokens and the GitHub repositories with their deploy keys. Everything is operated through a `justfile` from the MacBook; the hosts pull their own configuration from the private GitHub repo and update themselves overnight.

Short answer to the question "*Did you wire NixOS and OpenTofu together somehow?*": **Yes, but deliberately loosely – through files in the repo, not through a shared tool.** OpenTofu writes sops-encrypted files into `hosts/<host>/` that the NixOS modules read at build time, and both sides read the same service list `hosts/grammi/proxy-services.json`. No terranix, no Tofu run from within Nix. Details in NixOS & OpenTofu.

Audience: people who can read NixOS flakes, sops-nix and Terraform/OpenTofu.

NixOS & OpenTofu

Yes, the two are wired together – with the repo as the shared interface. OpenTofu manages what exists outside the machines behind an API (NetBird account, deSEC tokens, Pocket ID groups); NixOS manages the machines. The two never talk to each other directly: Tofu only runs on the MacBook through `just`, Nix never evaluates Tofu state.



Tofu never writes to a host directly: every result ends up as a file in the repo and reaches the host only with the next deploy.

The touch points

Direction	File	Created by	Read by	Contents
Tofu → NixOS	`hosts/<host>/netbird.yaml`	`just netbird-key <host>` (`tofu output setup_keys` → `sops encrypt`)	`modules/netbird.nix` (`sopsFile`)	One-time setup key, valid for 24 h
Tofu → NixOS	`hosts/<host>/desec.yaml`	`just desec-key <host>` (`tofu output acme_tokens` → `sops encrypt`)	`modules/acme.nix` (`sopsFile`)	deSEC token for TXT records only, VMs pinned to their IP
Tofu → NixOS	`terraform/netbird/expected-peers.json`	`just netbird-apply` (`tofu output expected_peers`, computed from the policies in `expected-peers.tf`)	`hosts/grammi/netbird-expected-peers.nix` (`builtins.readFile`)	Expected NetBird peers per host as metric `netbird_expected_peers`, next to the real network map in the NetBird dashboard (e.g. aldrin 6, grammi 8)

Direction	File	Created by	Read by	Contents
Tofu → NixOS	`terraform/netbird/admin-peers.json`	`just netbird-apply` (`tofu output admin_peers`, from the group `admin-workstations`, `admin-peers.tf`)	`flake.nix` (`builtins.readFile`, option `services.docsSite.allow`)	Mesh addresses of the admins' devices – who may read the private docs
shared	`hosts/grammi/proxy-services.json`	by hand	`terraform/netbird/reverse-proxy.tf` (`jsondecode(file(...))`) and `topology.nix`	Name → app, port, groups of the reverse proxy services
repo → both	`hosts/<host>/github.yaml` + `github-deploy.pub`	`just github-key <host>` (`ssh-keygen` → `sops encrypt`)	private half: `modules/deploy.nix` (`sopsFile`); public half: `terraform/github` (`file(...)`)	Deploy key for `/etc/nixos`; Tofu only ever sees the public half
NixOS → Tofu	NetBird server on grammi	`hosts/grammi/netbird.nix`	provider via `https://mesh.homelab.example`	The API Tofu runs against in the first place
NixOS → Tofu	Bootstrap tokens	`just netbird-setup-owner`, `netbird-proxy-token` (SSH to grammi, server API)	`terraform/netbird/secrets.yaml` and `hosts/grammi/netbird-proxy.yaml`	Owner PAT, proxy token

The flow for a new host: Tofu creates the setup key, `just netbird-key` writes it encrypted for exactly that host (age key derived from its SSH host key) to `hosts/<host>/netbird.yaml`, commit, deploy within 24 h. The host registers itself on start; Tofu does not manage peers, it looks them up by name via `data "netbird_peer"`.

What is deliberately not wired

- **No shared host list.** Hosts are listed three times: `nixosConfigurations` in `flake.nix`, `local.hosts` in the Tofu modules and `hosts :=` in the `justfile`. A new host has to be added in all three places.
- **No terranix / no Tofu in the Nix build.** Tofu state is local (gitignored, in the main checkout, also from worktrees) because it holds setup keys and tokens in plain text.
- **No provisioning of the machines.** The VMs are created by hand at the provider, ursalia/cuckoosegg following `docs/install-*.md`.
- **Public DNS** lives in DNSControl (its own repo, only linked in via symlink), not in Tofu.

So the line is: whatever Nix can build deterministically from the repo, Nix does. Whatever is created once through an API and returns a secret, Tofu does – and hands it over encrypted as a file.

Secrets, network & access

Secrets (sops-nix, age)

Every host file is encrypted for exactly two recipients: the age key derived from the host's SSH host key and the offline recovery key `admin`. So the MacBook cannot read host secrets – `just secrets <host>` edits them on the host and fetches the result back. Encrypting for a host, on the other hand, works anywhere (only the public key is needed); that is exactly what `netbird-key` and `desec-key` use.

File	Recipients	Contents
`hosts/<host>/secrets.yaml`	host + admin	Deploy key, app passwords, OIDC clients, restic
`hosts/<host>/netbird.yaml`	host + admin	NetBird setup key (from Tofu)
`hosts/<host>/desec.yaml`	host + admin	deSEC token (from Tofu)
`hosts/<host>/github.yaml`	host + admin	GitHub deploy key (from `just github-key`)
`terraform/*/secrets.yaml`	workstation + admin	API tokens for the providers

The workstation key lives in 1Password and is read via `SOPS_AGE_KEY_CMD`, never as a file. `env.sh` in each Tofu module decrypts the tokens only into the shell of the respective recipe.

Network

- **NetBird mesh** (`192.0.2.0/24`, peer names under `peer.homelab.example`): control plane on grammi, login via Pocket ID. Private range (RFC 1918) instead of CGNAT, because tailscaled drops packets from 100.64/10 while both clients run in parallel.
- **Groups:** `servers` (all hosts except cuckoosegg), `subraum-devices`, `admin-workstations` (MacBook), plus person groups from Pocket ID.
- **Home LAN** (`198.51.100.0/24`): routed through ursalia (metric 100), with the two Apple TVs as fallback (group `home-routers`, metric 200, own setup key via `just netbird-router-key`, owned by nobody, in no policy); for group `home-lan`, not for devices that are already at home (posture check `not-at-home`). The Apple TVs need "Quantum-Resistance" on in the app, otherwise no peer (Rosenpass strict) accepts a tunnel to them.
- **ursalia's uplink:** WLAN into the home LAN's main network (Intel BE200, `hosts/ursalia/wifi.nix`): wpa_supplicant, WPA3 with WPA2 fallback, country DE, all bands, passphrase via sops. Its final spot has no cable; eno1 keeps DHCP as a fallback with a worse route metric, so the WLAN stays the primary link when both are up.
- **zorlock's uplink:** WLAN as well (`en1`, Broadcom); `hosts/zorlock/wifi.nix` makes sure `homelab` is a preferred network (`networksetup`, only ever adds it, passphrase via sops in `hosts/zorlock/wifi.yaml`). nix-darwin has no Wi-Fi module; band choice stays with macOS.
- **Subraum:** routed through cuckoosegg for `subraum-access`. cuckoosegg itself may only reach grammi's ingest (TCP 8428/9428) and may only be reached from the MacBook via SSH.
- **Tailscale** still runs in parallel during the migration and serves as break-glass: `NIXLAB_DOMAIN=example-tailnet.ts.net just deploy zummi`.

Access

Only 22/80/443 are open to the public. `modules/ssh.nix` allows root SSH only from mesh or Tailscale addresses; every host has its own login keypair. The mesh is IPv4 only for now; dual stack is listed as an open item.

Private docs: admin devices only

The private version of this documentation (`doku.homelab.example`, on aldrin and ursalia) contains every internal address and describes how the lab is run. That is more than every device in the mesh needs to see. So nginx serves it only to the mesh addresses of the admins' devices; everyone else gets 403 – with a page of its own that says why, which address the request came from and how a device gets in (without the navigation, so the page titles stay hidden). The name itself resolves for all peers.

From outside the same name leads to the public, anonymised docs (GitHub Pages): publicly `doku` is a CNAME to the reverse proxy, whose service `doku` hands it on without a login to a listener on aldrin (port 8091, mesh only) that knows nothing but a redirect. Whoever is in the mesh gets to aldrin and ursalia directly through the NetBird zone.

Who	Path	Result
Outside, or NetBird off	reverse proxy (grammi) → aldrin:8091	301 to the public docs
In the mesh, device in `admin-workstations`	straight to aldrin/ursalia	private docs
In the mesh, any other device	straight to aldrin/ursalia	403 page

Why through the proxy: this way `doku` is listed with the other public names in the NetBird dashboard instead of being a public vhost of its own on aldrin. The proxy cannot redirect by itself, only forward – hence the redirect listener. It has a port of its own rather than the docs vhost recognising the proxy by its address: the proxy is a peer of its own whose mesh address Tofu does not know. Linking `homelab.example` to the public docs is accepted on purpose; the real addresses stay replaced there.

What counts as an admin device: the devices in the group `admin-workstations` (`terraform/netbird/subraum.tf`), today the MacBook – the same device that may also SSH to the Subraum devices. Deliberately not the person group `homelab_admins`: that one is filled from Pocket ID with *all* devices of an admin user. Today that would be none (Calla has no devices of its own), and a phone signing in under an admin user would be in at once. An admin here is a device named in Tofu, not a login. A further device in `admin-workstations` therefore also gets SSH to the Subraum devices – the same trust. Tofu reads the members on every `just netbird-apply` and writes name → address to `terraform/netbird/admin-peers.json`; `flake.nix` hands the addresses to `services.docsSite.allow`. A new admin device therefore gets in after `just netbird-apply`, a commit and a deploy of aldrin and ursalia.

Why nginx and not a NetBird policy: `to-servers` lets the devices of all persons (`homelab` and `homelab_admins`) reach all ports on all servers. NetBird policies can only allow, never deny – a narrower rule for the docs would change nothing as long as `to-servers` stays as it is. Narrowing `to-servers` to a port list per service would work, but touches every service on every server for one static site. Moving the docs to a port of their own fails at the same point. So the policy still decides which hosts a device reaches, and nginx decides who gets the docs.

Why no login (Pocket ID): you read the docs when something is broken. A login would tie them to zummi (Pocket ID) and, through the reverse proxy, to grammi – exactly the hosts they are meant to be independent of. An address list needs nothing at runtime but nginx on the host itself.

What this deliberately accepts:

- The list is as current as the last `just netbird-apply` followed by a deploy. Until then a new admin device sees 403, and a removed device still gets in.
- The address stands for the device, not the person: whoever uses an admin device reads the docs.
- The list must never be empty – `just netbird-admin-peers` refuses an empty output, and the module fails the build with an assertion instead of quietly giving everyone 403.
- IPv4 only, like the mesh. With the IPv6 overlay the v6 addresses go into the same list.

Paperless: share links from outside

Paperless (`dokumente.homelab.example`, ursalia) is reachable only in the mesh. Whoever uses the app needs the NetBird client – every person. Until 2026-10-20 the full app was also public through the reverse proxy behind SSO; that is replaced.

From outside the same name serves share links only, behind a PIN: publicly `dokumente` is a CNAME to the reverse proxy, whose service `dokumente` hands it on after the PIN to a listener on ursalia (port 8092, mesh only, HTTP through the tunnel). That listener knows only `share/<slug>` (documents and bundles, both paperless' `SharedLinkView`) and 404 otherwise. An expired or unknown link makes paperless redirect to its login page; the listener answers that with a short note.

Who	Path	Result
Outside, or NetBird off	reverse proxy (grammi, PIN) → ursalia:8092	`/share/<slug>` only, 404 otherwise
In the mesh	straight to ursalia:443	full app (Pocket ID login), share links without PIN

Why the same name: paperless builds a share link from the origin of the UI. A link created in the mesh, `https://dokumente.homelab.example/share/<slug>`, therefore works for third parties without rewriting. A name of its own (say `teilen`) would mean editing every link by hand.

Why the listener filters, not the PIN: the PIN is a second hurdle in front of the unguessable slug, not a replacement for it. That the app itself is out of reach rests on nginx on ursalia alone. The proxy service has no path filter (target `path`), otherwise the redirect to `/accounts/login/` for expired links would no longer reach the listener.

PIN: exactly 6 digits (the NetBird API accepts nothing else), one for all shares. New one: `just dokumente-share-pin` (random, into `terraform/netbird/secrets.yaml`), then `just netbird-apply`; show it: `just dokumente-share-pin-show`. The proxy blocks a client address for 15 minutes after 5 wrong PINs within 5 minutes.

What this deliberately accepts:

- Whoever knows the PIN gets every link whose slug they know – no others. So create shares in paperless with an expiry date.
- CrowdSec does not see PIN attempts (TCP passthrough, no HTTP log on grammi), as with every proxy name; the proxy's own block remains.
- ursalia's nginx logs the proxy's mesh address as the client, not the visitor's (no `realip`: it would rewrite the address before `allow`/`deny`).

Deployment & operations

The pull hosts (zummi, aldrin, grammi, ursalia) keep `/etc/nixos` as a checkout of the private GitHub repo and build themselves. cuckoosegg is pushed to, zorlock is switched locally with `darwin-rebuild`.

Host	Manual deploy	Automatic
zummi, aldrin, grammi, ursalia	`just ship <host> "<message>"` = commit, push, `git pull` + `nixos-rebuild switch` on the host	nightly auto-upgrade, reboot window 04:00–06:00
cuckoosegg	`just deploy-push cuckoosegg` (built on ursalia, copied to the target)	ursalia pushes daily at 06:30
zorlock	`just deploy-zorlock` (`darwin-rebuild switch` with sudo)	none

Nightly upgrade

`modules/auto-upgrade-git.nix` updates `nixpkgs`, `nixpkgs-unstable` and `sops-nix`, builds, switches and pushes the new `flake.lock` back to GitHub with the host's deploy key. The GitHub host key is pinned because nobody is watching. A configuration error that only shows up on a cold boot therefore takes the host down at 04:00 – for the VMs there is the provider's console for that.

Monitoring

Everything ends up on grammi: **vmagent** on every host sends metrics to VictoriaMetrics (`:8428`), **Vector** sends journald and nginx logs to VictoriaLogs (`:9428`), both over the mesh. Grafana (`metrics`) and logchef (`logs`) sit behind the reverse proxy with SSO. Exporters per host come from `modules/metrics.nix`; whatever has no endpoint of its own goes through node_exporter's textfile collector (`/var/lib/prometheus-node-exporter-text-files`), written by small timers or services:

Textfile metric	Host	Source
`tailscale_*`	hosts with Tailscale	`tailscale metrics write`, every 30 s
`fwupd_updates_*`	ursalia	`fwupdmgr get-updates` after each metadata refresh
`wifi_link_*`	ursalia	`iw dev wlp2s0f0 link` (PHY mode, width, MCS, NSS), every 30 s
`node_wifi_*`, `wifi_link_*`	zorlock	`system_profiler SPAirPortDataType -json` every 60 s, same names as on ursalia (no receive rate, retries or separate receive PHY mode on macOS; macOS redacts SSID/BSSID without Location Services even for root, shown as "n/a" in the dashboard), plus `wifi_link_noise_dbm`
`netbird_expected_peers`	grammi	`terraform/netbird/expected-peers.json`, baked into the system at build time

The dashboards are JSON in `hosts/grammi/dashboards/`, provisioned from the repo (changes only via commit + deploy of grammi). `topology.json` is generated by `just topology`.

Dashboard	Shows
Hosts Overview	Scrape targets, CPU/memory/disk/network per host, failed units, certificates, backups (btrbk, restic), pending firmware, WLAN band/signal
Topology	Hosts, exporters and data flows as a Graphviz diagram
NetBird	Control plane reachable per host; peers per host as expected (Tofu) / known (network map) / connected P2P / connected relay, plus deviations; connection history, P2P latency, sync and setup times; netbird-server (signal, relay, gRPC, API)
WLAN	ursalia's and zorlock's band (2.4 / 5 / 6 GHz) over time, channel, signal, PHY mode, link rate, traffic, MCS, retries, time associated
Disk Health	SMART/NVMe of ursalia: status, critical warnings, wear, spare, temperature, data written
Paperless	Health (Celery, Redis, DB, index, classifier), documents, inbox, failed tasks, storage, log warnings
paperless-gpt	OCR and suggestion backlog, OCR time per page, LLM requests, durations, tokens, errors; zorlock's GPU and memory
Web & Security	nginx and Traefik requests, status codes, durations; CrowdSec bans, alerts, parser hits
Garage (S3 storage)	aldrin's Garage: cluster health, disk, S3 requests/errors/duration, resync and GC queues
VictoriaMetrics & VictoriaLogs	Ingest rate, disk usage, query duration, active series
zorlock (Mac mini)	mactop: thermal state, CPU/GPU, power, temperatures, memory, fan; Ollama requests, speed, model load times

Expected vs. known peers (NetBird): "expected" is computed by OpenTofu from the policies with the group membership at the last `just netbird-apply` (example: aldrin 6, grammi 8, cuckoosegg 4). "known" is what the management server really distributes. A device that joins later raises known at once, expected only after the next apply + deploy of grammi – on purpose, so a new peer is visible. The model ignores posture checks; see the header of `expected-peers.tf`.

Grafana pitfall: state timelines with `mergeValues` and colour mode "thresholds" turn numbers into threshold buckets ("1+", "< 1") before value mappings are applied. Use colour mode "fixed" and put text and colour into the mappings.

Add a new host

A host appears in several places in the repo, because NixOS and OpenTofu deliberately share no host list (see What is deliberately not wired). This guide walks through them in order.

1. Provide the machine

Rented VMs are created by hand at the provider; bare metal and the Subraum VM follow the installation guides in `docs/install-*.md`. The result is a NixOS install with an SSH host key – the host's age key is derived from it next.

2. Create the configuration

- Create `hosts/<host>/configuration.nix` and `hardware-configuration.nix`.
- Add it to `nixosConfigurations` in `flake.nix` with an explicit module list. Whatever is left out on purpose gets a comment saying why.

3. Add it wherever hosts are listed

File	Entry
`flake.nix`	`nixosConfigurations.<host>`
`inventory.json`	`hosts.<host>` with `"managed": true` – kind, site, owner, unlock (see Inventory)
`justfile`	`hosts := "…"` (pull hosts only, for `ship-all`)
`terraform/netbird/main.tf`	`local.hosts`: host → setup key group (`servers` or similar)
`terraform/desec-tokens/main.tf`	`local.hosts`: allowed source addresses, `acme_names`
`terraform/github/main.tf`	`local.hosts`: deploy key read-only or with write access

4. sops recipient

Derive the age key from the new host's SSH host key (`ssh-to-age` on `/etc/ssh/ssh_host_ed25519_key.pub`) and add it to `.sops.yaml`: an anchor under `keys` and a `creation_rule` for `hosts/<host>/(secrets|netbird|desec|github)\.yaml$` with the host and `admin` as recipients. The public host key itself goes into `inventory.json` (`sshHostKeys`, or later via `just inventory-collect`); `checks.inventory-sops` compares the two.

5. Mint keys and tokens

Recipe	Result	Note
`just netbird-key <host>`	`hosts/<host>/netbird.yaml`	one-off setup key, **valid for 24 h** – deploy right after
`just desec-key <host>`	`hosts/<host>/desec.yaml`	only for hosts with their own certificates
`just github-key <host>`	`hosts/<host>/github.yaml` + `github-deploy.pub`	pull hosts only

Recipes that need sops on the MacBook run with the 1Password sign-in in front (see just recipes).

6. Deploy and check

- First deploy within 24 h of `netbird-key`. The host registers itself in the mesh at startup; Tofu finds it by name through `data "netbird_peer"`.
- Run `just topology` and commit the result – otherwise `nix flake check` fails.
- `just inventory-collect` (host key, interfaces), set `network/ipv4` by hand, then `just inventory-test`; `just known-hosts` on the MacBook.
- Add app secrets once the host is running: Add secrets.
- Update the docs: Hosts & roles, Services.

Publish a new service

Services are named by function (`rezepte`, not `mealie`); new web services on grammi use PostgreSQL. First decide how the service should be reachable – that decides which files change.

Reachability	For	Examples
Mesh only	admin UIs, services for your own devices	`s3-admin`, `llm`, `dokumente`
Public behind the reverse proxy with SSO	web apps that should work on the road without the NetBird app	`rezepte`, `grist`, `metrics`
Directly public	services with their own login, for machines or everyone	`auth`, `s3.aldrin`

Mesh only

1. One line under `local.mesh_services` in `terraform/netbird/mesh-services.tf`: name → host(s). This creates a NetBird DNS zone of its own with A record(s) on the mesh address(es); public DNS does not know the name.
2. `just netbird-apply` – since NetBird 0.81 new zones reach the clients without a restart.
3. On the host an nginx vhost with `enableACME`, `acmeRoot = null` (DNS-01 via deSEC) and `allow 192.0.2.0/24; deny all;`.
4. If the service runs on ursalia or zorlock: add the name to the host's `acme_names` in `terraform/desec-tokens/main.tf` and run `just desec-apply` – their tokens may only write exactly those TXT records.
5. Deploy the host, `just topology` (new vhost).

Public behind the reverse proxy

For services on grammi:

1. An entry in `hosts/grammi/proxy-services.json`: name → app, port, Pocket ID groups. Tofu (`reverse-proxy.tf`) and `topology.nix` read the same file.
2. The service listens on all addresses; only 80/443 are open publicly, `nb-mesh` is trusted.
3. Push a CNAME `<name>` → `proxy.homelab.example` in DNSControl – a few minutes before the apply, otherwise the resolver caches NXDOMAIN.
4. Deploy grammi first, then `just netbird-apply`. At the end it runs `just netbird-proxy-certs`: it requests every new name once from outside so the proxy fetches its certificate, then restarts `netbird-proxy` once – otherwise the dashboard hangs at "Issuing certificate…" (NetBird discussion #7118).

Services on other hosts get a resource of their own in `terraform/netbird/reverse-proxy.tf` with the host as peer target (examples: `dokumente` on ursalia – share links only, behind a PIN –, `doku` on aldrin). Here too: push the CNAME first, `just netbird-apply` takes care of the certificates.

Directly public

nginx vhost on the host (ACME via DNS-01), A/AAAA records in DNSControl, CrowdSec reads the nginx logs. Only for services that have a login of their own anyway.

Afterwards

- Docs: Services, and Contents of hosts/<host>/ if new files appear there.
- Dual stack wherever possible (A and AAAA).

Add secrets

App passwords, OIDC clients and the like live in `hosts/<host>/secrets.yaml`, encrypted for the host and the recovery key `admin`. The MacBook cannot decrypt these files – that is why they are edited on the host (background: Secrets, network & access).

Steps

1. `just sync <host>` – otherwise the host has an outdated `secrets.yaml`, and `pull-secrets` brings it back into the repo.
2. `just secrets <host>` – opens sops on the host (with its host key, no 1Password sign-in) and afterwards fetches the result into the repo via `pull-secrets`.
3. Always quote values: `key: "value"`, single quotes if the value contains `"` or `\`. An unquoted value may be read by YAML as a number or bool; sops-nix then aborts `setupSecrets` and **no** secret of that generation gets installed.
4. Add the NixOS side (`sops.secrets."<name>"`) in the host module, commit, deploy: `just ship <host> "<message>"`.

What is not edited by hand

File	Written by
`hosts/<host>/netbird.yaml`	`just netbird-key <host>`
`hosts/<host>/desec.yaml`	`just desec-key <host>`
`hosts/<host>/github.yaml`	`just github-key <host>`
`terraform/*/secrets.yaml`	sops on the MacBook with the workstation key from 1Password

Everything that needs sops on the MacBook runs with the sign-in in front – see just recipes.

Rotate keys

What can be renewed how, and what to do afterwards. The recipes replace the old key at once – so commit and deploy right after.

Key	Recipe	Afterwards	When
A host's NetBird setup key	`just netbird-key <host>`	deploy within 24 h	after a reinstall or lost client state

Key	Recipe	Afterwards	When
A host's deSEC token	`just desec-key <host>`	deploy at once, then `just desec-verify`	on suspicion, otherwise never
A host's GitHub deploy key	`just github-key <host>`	commit + deploy	on suspicion
Tofu token for NetBird (service user `terraform`)	`just netbird-renew-token`	–	yearly
App secrets	`just secrets <host>` (see Add secrets)	deploy	on suspicion

SSH host key (= sops recipient)

A host's age key is derived from its SSH host key. A new host key therefore also means:

1. Derive the new age key with `ssh-to-age` and replace it in `.sops.yaml`.
2. Re-encrypt the host's files with `sops updatekeys` – this needs a key that can still read them (the old host key or `admin`).
3. Put the new public key into `inventory.json`: `just inventory-collect --force <host>`. `nix flake check` (`checks.inventory-sops`) fails until the inventory and `.sops.yaml` agree.
4. Commit and deploy before the old key is gone – every host's known_hosts comes from the inventory. On the MacBook: `just known-hosts`.

After a rotation

- Delete old keys at the other end (GitHub, deSEC and NetBird are handled by the recipe through Tofu).
- `just docs` if names or procedures changed.

Repository layout

One flake, `nixosConfigurations` for five hosts plus `darwinConfigurations.zorlock`. Every host gets an explicit module list in `flake.nix`; whatever is missing is missing with the reason in a comment next to it.

Path	Contents
`flake.nix` / `flake.lock`	Inputs, host definitions, packages, `lib.topology`, `lib.inventory`, `checks`
`modules/`	17 shared modules: base, nix, ssh, firewall, networking (ifstate), acme, crowdsec, tailscale, netbird, metrics/metrics-shipping, logging/log-shipping, qemu-guest, deploy, auto-upgrade-git, docs-site
`hosts/<host>/`	`configuration.nix`, hardware, service modules, sops files (`secrets.yaml`, `netbird.yaml`, `desec.yaml`)
`pkgs/`	Own packages: garage-ui, logchef, paperless-gpt, prometheus-paperless-exporter, ollama-metrics-proxy
`terraform/netbird/`	OpenTofu: NetBird account, policies, DNS zones, reverse proxy services, people/groups (incl. Pocket ID)
`terraform/desec-tokens/`	OpenTofu: narrowly scoped deSEC tokens per host for DNS-01
`terraform/github/`	OpenTofu: the repositories `nixlab` (private) and `nixlab-docs` (public), deploy keys per host
`topology.nix`	Generates the Grafana topology dashboard from the host configs
`inventory.json` / `inventory.nix`	Lab inventory (every machine: kind, site, uplink, relations, unlock) and its validation, see Inventory

Path	Contents
`inventory.schema.json`	JSON Schema for editing `inventory.json`, generated
`tests/`	Tests evaluated by `nix flake check` (`inventory.nix`)
`justfile`	All operational workflows: deploy, secrets, netbird-*, desec-*
`docs/`	This project documentation: pages under `site/{de,en}/`, Zensical config, build (`build.sh`, `site-src.sh`, `assemble.py`, `gen-reference.py`), the private site's package (`package.nix`) and what it would otherwise load from third parties – fonts, glightbox (`assets.nix`), install guides (ursalia, cuckoosegg), drafts of upstream issues
`dnsconfig.js`	Symlink into the separate DNSControl repo – public DNS records live there
`.sops.yaml`	age recipients per host, `admin` (offline recovery) and `workstation` (1Password)

Where do I change what?

Which file is responsible for which change, how often that happens, and what to do afterwards.

What	Where	How often	Afterwards
A service, package or option on one host	`hosts/<host>/*.nix`	often	`just ship <host> "…"`
Something for all hosts	`modules/*.nix`	now and then	`just ship-all "…"`; new module: public or in `docs/modules-private.txt`?
A name behind the reverse proxy (grammi)	`hosts/grammi/proxy-services.json`	rarely	CNAME in DNSControl, deploy grammi, `just netbird-apply`
A mesh-only name	`terraform/netbird/mesh-services.tf`	rarely	`just netbird-apply`
Who may read the private docs	group `admin-workstations` (`terraform/netbird/subraum.tf`, peer names)	rarely	`just netbird-apply`, commit `admin-peers.json`, deploy aldrin and ursalia
NetBird policies, groups, people, routes	`terraform/netbird/*.tf`	rarely	`just netbird-plan`, `just netbird-apply`
Public DNS records	DNSControl repo (`dnsconfig.js`)	rarely	push there
What a deSEC token may do	`terraform/desec-tokens/main.tf`	rarely	`just desec-apply`
Repositories, deploy keys	`terraform/github/`	rarely	`just github-plan`, `just github-apply`
Version of an automatically updated input	`flake.lock`	nightly, automatic	–
Version of a separately pinned input	`flake.lock` (`nix flake update <input>`)	by hand	deploy the hosts that use it
Grafana dashboard	`hosts/grammi/dashboards/*.json`	now and then	deploy grammi
Topology (new host, exporter, vhost)	generated by `topology.nix`	when needed	`just topology`, commit

What	Where	How often	Afterwards
Machine data: kind, hardware, site, uplink, power, unlock, networks, interfaces, host keys	`inventory.json`	with every new device	`just inventory-collect`, `just inventory-test` (Inventory)
Inventory format (fields, kinds)	`spec` in `inventory.nix`	rarely	`just inventory-schema`, test case, docs
A host's secrets	`hosts/<host>/secrets.yaml`	rarely	Add secrets
This documentation	`docs/site/{de,en}/`	with every change	`just docs`

Contents of hosts/<host>/

What a host directory contains and who writes it. Not every host has every file.

File	Content	Written by
`configuration.nix`	host configuration: network, services, vhosts	by hand
`hardware-configuration.nix`	hardware, file systems	`nixos-generate-config`, then by hand
`disk-config.nix`	partitioning (disko, ursalia only)	by hand
further `*.nix`	one service per file, e.g. `backup.nix`, `samba.nix`, `netbird.nix`, `traefik.nix`	by hand
`secrets.yaml`	app secrets, encrypted for host + admin	`just secrets <host>`
`netbird.yaml`	NetBird setup key	`just netbird-key <host>` (Tofu)
`desec.yaml`	deSEC token for DNS-01	`just desec-key <host>` (Tofu)
`github.yaml`	private half of the deploy key	`just github-key <host>`
`github-deploy.pub`	public half, read by `terraform/github`	`just github-key <host>`
further `*.yaml`	sops files of single services, e.g. `netbird-server.yaml`, `netbird-proxy.yaml` (grammi), `push-cuckoosegg.yaml` (ursalia)	their recipes, e.g. `just netbird-server-secrets`, `just netbird-proxy-token`
`push-key.pub`	public key for the push deploy (cuckoosegg only)	`just cuckoosegg-push-key`
`dashboards/*.json`	Grafana dashboards (grammi only)	by hand, `topology.json` via `just topology`
`proxy-services.json`	services behind the reverse proxy (grammi only)	by hand, read by Tofu and `topology.nix`

Inputs and their pins

Only `nixpkgs`, `nixpkgs-unstable` and `sops-nix` move with the nightly auto-upgrade (`updateInputs` in `modules/deploy.nix`). Anything that would lock the whole lab out on a broken release is pinned separately and bumped by hand.

Input	Branch / tag	Used by	Nightly?
`nixpkgs`	nixos-26.05	all NixOS hosts	yes
`nixpkgs-unstable`	nixos-unstable	grammi (Grist), ursalia (Paperless 3.x), zorlock (mactop)	yes

Input	Branch / tag	Used by	Nightly?
`sops-nix`	main	all	yes
`nixpkgs-netbird`	nixos-unstable	NetBird server and all clients	no – server first, then clients
`nixpkgs-pocketid`	nixos-26.05	zummi (pocket-id package)	no
`disko`	main	ursalia (all mounts)	no
`lanzaboote`	v1.1.0	ursalia (Secure Boot)	no
`nixpkgs-darwin`, `nix-darwin`	26.05-darwin	zorlock	no (no auto-upgrade)

`nix flake check` additionally verifies that `hosts/grammi/dashboards/topology.json` matches what `topology.nix` currently generates – a new exporter without `just topology` fails the check.

Hosts & roles

Three rented VMs carry the public services, ursalia and zorlock sit at home, cuckoosegg in the Subraum hackerspace. All of them are reachable in the NetBird mesh as `<host>.peer.homelab.example`; root SSH exists only there. Kind, site, uplink and disk unlock of every machine – including the routers, KVM and appliances – are in the inventory.

Host	Platform	Role	Notes
grammi	NixOS, rented VM	NetBird server (`mesh.homelab.example`) + reverse proxy, Traefik, monitoring (VictoriaMetrics/-Logs, Grafana, logchef), Mealie, Grist, PostgreSQL	Central sink for metrics and logs of all hosts
zummi	NixOS, rented VM	Pocket ID (`auth.homelab.example`) – login for all services and NetBird	pocket-id pinned separately
aldrin	NixOS, rented VM	Garage S3 (`s3.aldrin.homelab.example`), garage-ui (`s3-admin`), backup target for restic, docs site (`doku`)	Forked garage-ui package with subpath support
ursalia	NixOS, bare metal (ThinkCentre M720q)	Paperless-ngx 3 + paperless-gpt (`dokumente`), docs site (`doku`), Samba for the scanner, routing peer into the home LAN, build host for cuckoosegg	LUKS + btrfs via disko, TPM unlock, Secure Boot (lanzaboote), WLAN (Intel BE200), no CrowdSec
cuckoosegg	NixOS, VM on the Subraum Proxmox	NetBird routing peer into the Subraum networks	Group `subraum-devices` instead of `servers`, no repo/deploy key, deployed by push
zorlock	macOS + nix-darwin, Mac mini M4	Ollama on the Apple GPU (`llm`), used by paperless-gpt	None of the `modules/*.nix`; FileVault unlock after reboot via LAN SSH

Inventory

`inventory.json` lists every machine of the lab, whether this flake configures it or not: what kind of device it is (vendor, model, serial), where it stands, how that place reaches the internet, what it runs on or is built into, where its power comes from, how its disk gets unlocked after a reboot, its networks, interfaces and addresses, and its SSH host keys. These are the facts a NixOS config does not hold. Today the host keys are used: every host's known_hosts comes from here, and `nix flake check` ties them to the age

keys in `.sops.yaml`. Later the rest is meant for grouping in the topology and for alert dependencies (an uplink is down, so don't page for every host behind it).

File	Role
`inventory.json`	The data. JSON, so Nix (`builtins.fromJSON`) and OpenTofu (`jsondecode`) can both read it without help.
`inventory.nix`	Format (`spec`), validation, derived view, JSON Schema generator. Exposed as `lib.inventory` in `flake.nix`.
`inventory.schema.json`	JSON Schema for the editor (completion, hover help, inline errors). Generated, never edited.
`tests/inventory.nix`	Tests for the validator: a complete example inventory and one case per rule.
`flake.nix`	`lib.inventory`, the checks, and `programs.ssh.knownHosts` for every host (NixOS and nix-darwin).

Rule of thumb for what goes in: **only what cannot be derived from a config.** OS, architecture, CPU/RAM, mesh address, exporters and backup targets stay out; `topology.nix` reads those from `nixosConfigurations`. Every relation is stored in **one direction only**; the reverse is derived, so the two cannot disagree.

Structure

Five sections, each an object of entries keyed by id. Ids are lower-case letters, digits and dashes (usable unchanged as Graphviz node or label values).

```
owners   <──────── hosts.owner
uplinks  <──────── sites.uplinks[]
sites    <──────── hosts.site, networks.site
networks <──────── hosts.interfaces.<name>.network
hosts    <──────── hosts.runsOn / inside / power / manages[] / unlock.tang[], networks.dhcpServer
```

A `"$schema": "./inventory.schema.json"` key at the top is allowed and ignored; it tells the editor where the schema is.

owners

Who administers a device: it decides whom to call and what may be changed.

Field	Type		Meaning
`name`	string	required	Display name
`contact`	string		Who to talk to when this owner's devices are affected
`note`	string		Free text

uplinks

An internet line. Several sites may share one (rarely), one site may have several (primary and backup).

Field	Type		Meaning
`isp`	string		Provider; for a data centre the hoster
`asn`	integer ≥ 1		AS number of the provider's network, without "AS"

Field	Type		Meaning
`medium`	`fiber` `dsl` `cable` `lte` `satellite` `datacenter`		Kind of line
`bandwidth`	`{ "down": int, "up": int }`		Booked bandwidth in Mbit/s, both values required
`ipv4`	`public` `cgnat` `none`		What the line gives for IPv4
`ipv4Static`	boolean		Fixed public IPv4 address (only meaningful with `public`)
`ipv6`	boolean		Native IPv6
`ipv6PrefixLength`	integer 1–128		Size of the delegated IPv6 prefix (56 for a /56); only with `ipv6`
`ipv6Static`	boolean		Fixed IPv6 prefix; only with `ipv6`
`note`	string		Free text

All fields are optional, so a line can be entered before its details are known.

sites

A place where devices stand.

Field	Type		Meaning
`city`	string	required	City
`kind`	`residential` `hackerspace` `datacenter`	required	Kind of place
`provider`	string		Who runs the place (hoster, club); empty for a home
`uplinks`	list of uplink ids	required, ≥ 1	Internet lines, **primary first**, backups after it
`note`	string		Free text

hosts

Every device with an address that matters: servers, VMs, routers, APs, KVMs, PDUs, appliances.

Field	Type		Meaning
`kind`	see kinds	required	What the device is
`vendor`	string		Manufacturer (Lenovo, Apple, GL.iNet). Not for `vm`/`container`
`model`	string		Model without the vendor (ThinkCentre M720q), or for a VM the provider's product/plan name
`productId`	string		The vendor's id for drivers and firmware: Lenovo machine type (`10T7…`), Apple model identifier (`Mac16,10`). Not for `vm`/`container`
`serial`	string		Serial number, unique lab-wide. Not for `vm`/`container`
`site`	site id	required*	Where it is. *Optional with `runsOn` or `inside`: then it is inherited
`owner`	owner id	required	Who administers it
`managed`	boolean	required	Configured by this flake – must match `flake.nix`

Field	Type		Meaning
`criticality`	`core` `service` `lab`	default `service`	`core`: the lab stops working without it; `service`: a service is down; `lab`: nothing depends on it
`availability`	`always-on` `intermittent`	default `always-on`	`intermittent`: offline is normal (laptop), not an incident
`unlock`	object, see Disk unlock		Leave it out for an unencrypted disk
`runsOn`	host id		Only `vm`/`container`: the machine it runs on
`inside`	host id		Physically built into another device
`power`	host id		Power source: PDU/UPS, PoE switch, or the host a card sits in
`manages`	list of host ids		Only `kvm`: machines it gives out-of-band access to
`sshHostKeys`	list of strings		SSH host public keys, `"<type> <base64>"` without comment, see SSH host keys
`interfaces`	object: name → interface		Network interfaces, see Interfaces
`note`	string		Free text

Kinds { #kinds }

Kind	For
`server`	Physical machine running services
`hypervisor`	Physical machine running VMs/containers for others
`vm`, `container`	Virtual; the only kinds that may have `runsOn`
`router`, `switch`, `ap`	Network (`ap` = WLAN access point)
`pdu`, `ups`	Power
`kvm`	IP-KVM, external or as a card inside its host; the only kind that may have `manages`
`nas`, `workstation`, `laptop`	As named
`appliance`	Anything else with an address: Apple TV, printer, …

Whether a host is virtual follows from `kind` (`vm`, `container`) – not from `runsOn`. A rented VM is a `vm` without `runsOn`: the provider's hypervisor is not in the inventory.

networks

A layer-2 network with its IPv4 prefix: the home LAN, a VLAN in the hackerspace.

Field	Type		Meaning
`name`	string	required	Display name (the id is the technical key)
`site`	site id	required	Where the network is
`vlan`	integer 1–4094		802.1Q VLAN id; leave out for an untagged network. Unique per site – another site may use the same id
`ipv4`	prefix, e.g. `192.0.2.0/24`		Without host bits
`ipv6`	prefix, e.g. `2001:db8:1::/64`		Only when it is fixed (ULA, static GUA) – not an ISP prefix that changes

Field	Type		Meaning
`dhcpServer`	host id		Serves DHCP for IPv4; must be at the site and have an interface in this network
`dhcpPool`	`{ "start": ip, "end": ip }`		Dynamic range; inside `ipv4`, start ≤ end. Needs `ipv4` and `dhcpServer`
`note`	string		Free text

A network without any interface is allowed – it can be entered before its devices are.

Interfaces { #interfaces }

`hosts.<id>.interfaces` maps interface names to their data. The name is what the OS calls it (`eno1`, `wlp2s0f0`, `en0`); for an appliance any label (`lan`, `wan`). A VLAN interface on a trunk (`eno1.20`) is an entry of its own; its VLAN id comes from its network.

```
"interfaces": {
  "eno1":    { "mac": "52:54:00:12:34:56", "medium": "ethernet", "linkLocal": "eui64",
               "network": "lan", "ipv4": { "mode": "reserved", "address": "192.0.2.10" } },
  "eno1.20": { "network": "iot", "linkLocal": "fe80::1234", "ipv4": { "mode": "dhcp" } }
}
```

Field	Type		Meaning
`mac`	`aa:bb:cc:dd:ee:ff`, lower case		Hardware address. Unique lab-wide, no multicast address
`medium`	`ethernet` `wifi`		Physical medium
`linkLocal`	`eui64` `stable-privacy` `random` or an `fe80::` address		How the IPv6 link-local address is formed
`network`	network id		Network it is in; at the same site as the host
`ipv4`	`{ "mode": …, "address": … }`		IPv4 addressing, needs `network`
`note`	string		Free text

Link-local. Only with EUI-64 does the address follow from the MAC (`fe80::` + MAC with the U/L bit flipped and `ff:fe` in the middle); the derived `ipv6LinkLocal` is computed then. Many systems use `stable-privacy` instead (RFC 7217: stable, but not computable), macOS and tvOS do not derive it from the MAC either. When the address is stable and known, enter it directly (`"fe80::…"`); `just inventory-collect` does exactly that when it does not match EUI-64. `eui64` needs a `mac`; with a *locally administered* Wi-Fi MAC (private Wi-Fi address) it is an error, because that MAC – and with it the address – is usually random. A locally administered Ethernet MAC is fine (QEMU's `52:54:00:…` is one).

IPv4 – `ipv4.mode`:

Mode	Meaning	Rules
`reserved`	Fixed DHCP lease by MAC (reservation on `dhcpServer`)	`address` and `mac` required; the network needs a `dhcpServer`
`static`	Configured on the device itself	`address` required; outside `dhcpPool`
`dhcp`	Dynamic	no `address`

Every `address` lies inside the network's `ipv4` prefix, is neither network nor broadcast address (except /31, /32) and occurs once per network. A network without `ipv4` cannot take addresses.

IPv6 is in here only as link-local and as a fixed prefix per network: DHCPv6 reservations hang on the DUID, not the MAC, and an ISP prefix (SLAAC) changes.

SSH host keys { #ssh-host-keys }

`sshHostKeys` holds a device's public host keys – for managed and unmanaged devices alike – as `"<type> <base64>"` without the comment: `ssh-ed25519`, `ecdsa-sha2-nistp256/384/521` or `ssh-rsa`. One ed25519 key is enough; others only when the device has no ed25519 (Dropbear, KVMs). Checked: the format, that the key data matches its type (the base64 starts with the encoded type name), and that no key occurs twice – a cloned VM that kept its host key would show up that way.

Where they go:

Consumer	How
Every NixOS host and zorlock	`programs.ssh.knownHosts` from `lib.inventory.knownHosts` (in `flake.nix`, the module `inventoryKnownHosts` in every host's list). Deploys and ursalia's unattended push to cuckoosegg need no trust-on-first-use
The MacBook	`just known-hosts` writes `~/.ssh/known_hosts.nixlab`; ssh reads it with `UserKnownHostsFile ~/.ssh/known_hosts ~/.ssh/known_hosts.nixlab` under `Host *` in `~/.ssh/config`
`.sops.yaml`	`checks.inventory-sops`: for every managed host, `ssh-to-age` of its ed25519 key must give the age recipient `&<host>` in `.sops.yaml`. Catches a host key that changed (reinstall, rotation) without `sops updatekeys`, and a mistyped key. Hosts without a key yet are skipped

Names per key: the host's id, for managed hosts `<host>.peer.homelab.example` (the mesh name), and every fixed IPv4 address of its interfaces (`static`/`reserved`). One entry per key, named `<host>` or `<host>-<n>` with several keys.

Collecting from the devices

`just inventory-collect` fetches what is tedious to type:

- **Managed hosts**, over the mesh (root, zorlock as `admin`):
  - host keys via `ssh-keyscan` (only the ed25519 one when there is one);
  - per interface name, MAC, `ethernet`/`wifi` and link-local – `eui64` when the address matches the MAC, otherwise the address itself. On Linux every interface with a device behind it (hardware or virtio; no `lo`, mesh, tunnels, bridges, veths), on macOS the hardware ports without Thunderbolt;
  - for hardware that is not virtual: `vendor`, `model`, `productId`, `serial`. On Linux from `/sys/class/dmi/id` (Lenovo: model from `product_version`, machine type from `product_name`; everybody else: model from `product_name`), on macOS from `system_profiler` (model as "Mac mini (M4, 16 GB)", the model identifier as `productId`). Placeholders like "To be filled by O.E.M." are dropped.
- **Unmanaged devices**: only host keys, at their first fixed IPv4 address – enter `network`/`ipv4` first.

Call	Effect
`just inventory-collect`	All devices; **adds** only what is missing, a differing value is printed and kept
`just inventory-collect ursalia beryl`	Only these devices
`just inventory-collect --force ursalia`	Takes the device's value where it differs, and prints each value it overwrote

A changed host key should be a decision, hence `--force` – best with the host named. Even `--force` never touches what the device does not report: `network`, `ipv4`, notes, and interfaces it no longer has (remove those by hand).

Relations

Field	Meaning	May point at	Derived reverse
`runsOn`	Virtualisation	`vm` → `hypervisor`; `container` → `hypervisor`, `server`, `nas`, `vm`	`guests`
`inside`	Physical installation (PCIe card, module)	any host	`contains`
`power`	Power supply	any host	`powers`
`manages`	Out-of-band access (on the KVM)	any host but itself	`managedBy`
`unlock.tang`	Tang server for NBDE	any host but itself	`tangClients`
`interfaces.*.network`	Network membership	a network at the host's site	`networks.<id>.interfaces`
`networks.dhcpServer`	DHCP for a network	a host with an interface in it	–

`runsOn`, `inside` and `power` together form the dependency graph: what a host cannot work without. It must be free of cycles; the derived `upstream` of a host is its transitive closure. `runsOn` and `inside` exclude each other.

`inside` says nothing about power: whether a PCIe KVM card lives off its host's slot (standby power) or off PoE/USB is what `power` says. A card that only draws from the slot gets `"power"` set to its host. Then "host and KVM both gone" points at power or network, "host gone, KVM still there" at the OS – and the KVM is the way in.

Disk unlock

```
"unlock": { "methods": ["tpm", "nbde"], "threshold": 1, "tang": ["nas"], "fallback": "remote-
        ssh" }
```

Field	Type		Meaning
`methods`	list of `tpm`, `nbde`	required (may be empty)	Automatic methods. Empty: only a human can unlock
`threshold`	integer ≥ 1	default 1	How many methods must succeed – the Clevis `sss` threshold. 1 = any, 2 = TPM **and** Tang
`tang`	list of host ids	required with `nbde`	Tang servers; any one of them is enough
`fallback`	`remote-ssh` `console`		What a human does when the methods fail: SSH (initrd SSH, FileVault over SSH) or console/KVM

Case	`unlock`
Unencrypted	leave out
TPM, passphrase at the console as fallback	`{"methods": ["tpm"], "fallback": "console"}`
Only a human, over SSH (e.g. FileVault)	`{"methods": [], "fallback": "remote-ssh"}`
TPM **or** Tang	`{"methods": ["tpm", "nbde"], "tang": ["a"]}`

Case	`unlock`
TPM **and** Tang (disk only unlocks in its own network)	`{"methods": ["tpm", "nbde"], "threshold": 2, "tang": ["a"]}`
Only Tang, two servers	`{"methods": ["nbde"], "tang": ["a", "b"], "fallback": "console"}`

NBDE is a dependency of its own: a host that **needs** Tang (`nbde` without `tpm`, or `threshold` 2) only comes back once one of its Tang servers is up unattended itself. The validator follows these chains. Is there a host that would never come up without a human – two hosts that are each other's only Tang server, or a Tang server that itself waits at a passphrase prompt – that is an error. The derived `unattendedBoot` says per host whether it comes back after a reboot or power cut on its own.

Not checked: whether a Tang server actually runs Tang. That lives in its config, not here.

Validation

`nix flake check` (or just the inventory part: `just inventory-test`) validates in three stages. Each stage only runs when the one before found nothing, so a typo does not turn into twenty follow-up errors. Every error names the path:

1. Structure – top level first, on its own, then every entry:

- only `$schema`, `owners`, `uplinks`, `sites`, `networks`, `hosts` at the top, all five sections present
- valid ids and interface names, no unknown fields (catches typos), required fields present
- types, enums, minimums/maximums, formats (MAC, IPv4 address and prefix, IPv6 prefix, link-local, SSH key), no duplicates in lists
- references point at an existing entry of the right section

```
hosts.vps.critcality: unknown field (known: availability, criticality, inside, interfaces, kind,
managed, manages, model, note, owner, power, productId, runsOn, serial, site, sshHostKeys, unlock,
vendor)
hosts.nas.interfaces.eno1.mac: "52-54-00-12-34-56" is not a lower-case MAC address like
aa:bb:cc:dd:ee:ff
hosts.app.runsOn: "pve2" is not in hosts
uplinks.fiber-home.asn: expected a whole number, got "AS64496"
```

2. Rules between entries:

- `runsOn` only for `vm`/`container` and only on an allowed kind; not together with `inside`
- `manages` only for `kvm`; nothing points at itself
- `runsOn`/`inside`/`power` without cycles
- `site` present or inheritable; an explicit `site` matches the inherited one
- every site has an uplink; no uplink, site or owner without use; IPv6 details of an uplink only with `ipv6`
- `unlock`: at least a method or a fallback; `threshold` ≤ number of methods; `tang` exactly when `nbde`
- `managed` agrees with `flake.nix`: every `nixosConfigurations`/`darwinConfigurations` entry is in here as managed, and every managed entry has a configuration
- hardware: `vendor`/`productId`/`serial` not on `vm`/`container`; no serial twice
- SSH keys: key data matches the type, no key on two devices
- interfaces: network at the host's site; `ipv4` needs a network, `address` according to the mode, inside the prefix, not network/broadcast, static outside the pool, unique per network; `reserved` needs `mac` and a `dhcpServer`; MAC unique and not multicast; `eui64` needs a MAC and no private Wi-Fi MAC
- networks: prefix without host bits; pool inside the prefix, start ≤ end, only with `ipv4` and `dhcpServer`; `dhcpServer` at the site and with an interface in the network; VLAN id unique per site

```
hosts.app.runsOn: a vm cannot run on nas (kind nas; allowed: hypervisor)
hosts: runsOn/inside/power form a cycle through pve, ups1
hosts.laptop.managed: true, but flake.nix has no configuration of that name
hosts.router.interfaces.lan.ipv4.address: 192.0.2.150 is static but inside the DHCP pool of
network lan
networks.lan.vlan: 20 is also used by networks.iot at site home
hosts.router.sshHostKeys[0]: same key as hosts.nas.sshHostKeys[0] - a cloned machine that kept its
host key?
```

3. Unlock chains – see Disk unlock:

```
hosts.nas.unlock: never unlocks unattended - none of its Tang servers (vps) comes up without a
human
```

Separately, as its own check because it needs `ssh-to-age`: `checks.inventory-sops` (see SSH host keys).

```
hosts.cuckoosegg: host key gives age1z9rt…, .sops.yaml has age1z8uz… - host key changed without
sops updatekeys, or a wrong key in inventory.json
```

The schema in `inventory.schema.json` only covers stage 1 (without references) and is there for the editor; the validation in `inventory.nix` is what counts.

Derived view

`lib.inventory.inventory` (on the console: `just inventory`) is the validated inventory with everything derived. Every section gets its `id`; additionally:

Section	Derived
`owners.<id>`	`hosts` – the hosts it administers
`uplinks.<id>`	`sites` – the sites on this line
`sites.<id>`	`hosts` – everything there, inherited sites included
`hosts.<id>`	`site` (effective, inherited along `runsOn`/`inside`), `virtual`, `uplinks` (of its site, primary first), `guests`, `contains`, `powers`, `managedBy`, `tangClients`, `upstream` (transitive, nearest first), `unattendedBoot`
`hosts.<id>.interfaces.<name>`	`name`, `vlan` (from its network), `ipv6LinkLocal` (computed for `eui64`, as given for a fixed address, otherwise `null`)
`networks.<id>`	`interfaces` – every interface in it (`host`, `interface`, `mac`, `address`, `mode`); `reservations` – what `dhcpServer` has to hand out by MAC (`host`, `interface`, `mac`, `address`), sorted by address

Besides `inventory`: `lib.inventory.knownHosts` (the `programs.ssh.knownHosts` value) and `lib.inventory.knownHostsFile` (the same as known_hosts text).

Optional fields are always present in the derived view: defaults filled in (`criticality`, `availability`, `unlock.threshold`), otherwise `null` or `[]`. A consumer never has to write `or`.

If the inventory is invalid, `inventory` throws with **all** errors at once; `lib.inventory.errors` is the list itself (empty when valid).

Examples

The examples are the test inventory from `tests/inventory.nix`, written as JSON. It covers every field and relation.

Rented VM – virtual, the provider's hypervisor is not in the inventory:

```
"vps": { "kind": "vm", "site": "dc", "owner": "lab", "managed": true, "criticality": "core" }
```

VM on a hypervisor of others, which hangs on a UPS – `site` is inherited from `pve`:

```
"pve":  { "kind": "hypervisor", "site": "home", "owner": "club", "managed": false, "power":
        "ups1" },
"ups1": { "kind": "ups", "site": "home", "owner": "lab", "managed": false },
"app":  { "kind": "vm", "runsOn": "pve", "owner": "lab", "managed": true,
          "unlock": { "methods": ["nbde"], "tang": ["nas"] } }
```

Derived for `app`: `site` = `home`, `upstream` = `["pve", "ups1"]`, `unattendedBoot` = `true` (via the chain app → nas → vps).

NAS with TPM and Tang required together, plus a PCIe KVM card inside it:

```
"nas": { "kind": "nas", "vendor": "Example Corp", "model": "Example NAS", "productId": "EX-1000",
         "serial": "NAS0001", "site": "home", "owner": "lab", "managed": true,
         "unlock": { "methods": ["tpm", "nbde"], "threshold": 2, "tang": ["vps"] } },
"kvmcard": { "kind": "kvm", "inside": "nas", "power": "nas", "manages": ["nas"],
             "owner": "lab", "managed": false }
```

Derived for `nas`: `contains`, `powers` and `managedBy` = `["kvmcard"]`, `tangClients` = `["app"]`.

Laptop – offline is normal, only a human unlocks it:

```
"laptop": { "kind": "laptop", "site": "home", "owner": "lab", "managed": true,
            "availability": "intermittent", "unlock": { "methods": [], "fallback": "console" } }
```

Networks, a reservation and a router – the router hands out `nas`'s address by MAC; its own address is static, outside the pool:

```
"networks": {
  "lan": { "name": "Home LAN", "site": "home", "ipv4": "192.0.2.0/24", "ipv6": "2001:db8:1::/64",
           "dhcpServer": "router", "dhcpPool": { "start": "192.0.2.100", "end": "192.0.2.199" } },
  "iot": { "name": "IoT", "site": "home", "vlan": 20, "ipv4": "198.51.100.0/24" }
},
"router": { "kind": "router", "site": "home", "owner": "lab", "managed": false,
            "sshHostKeys": ["ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI…"],
            "interfaces": { "lan": { "network": "lan", "ipv4": { "mode": "static", "address": "192
        .0.2.1" } } } }
```

Derived: `networks.lan.reservations` = `nas/eno1` (192.0.2.10) and `app/ens18` (192.0.2.11); `nas.interfaces.eno1.ipv6LinkLocal` = `fe80::5054:ff:fe12:3456`; known_hosts names for `nas` = `nas`, `nas.peer.example`, `192.0.2.10`.

Site with primary line and LTE backup:

```
"uplinks": {
  "fiber-home": { "isp": "Example ISP", "asn": 64496, "medium": "fiber",
                  "bandwidth": { "down": 1000, "up": 500 }, "ipv4": "cgnat", "ipv6": true },
  "lte-backup": { "medium": "lte" }
},
"sites": {
  "home": { "city": "Exampletown", "kind": "residential", "uplinks": ["fiber-home", "lte-
        backup"] }
}
```

Reading it elsewhere. In Nix (another flake output, a module via `specialArgs`):

```
self.lib.inventory.inventory.hosts.app.upstream   # [ "pve" "ups1" ]
```

In OpenTofu – the raw data, without derived values and without validation (that stays with `nix flake check`):

```
locals {
  inventory = jsondecode(file("${path.module}/../../inventory.json"))
  servers   = [for id, h in local.inventory.hosts : id if h.managed]
}
```

Tests

`tests/inventory.nix` holds a complete, fictional inventory (the examples above) and 72 cases. Each case changes exactly one thing and states the expected errors – exact wording and order – or, for valid variants, values of the derived view. A case also fails if a broken inventory does not make `.inventory` throw.

Group	Covers
valid	Base inventory with all derived values; container on a VM; Tang loop broken by `threshold 1`; flake check switched off
uplinks	IPv6 prefix length out of range, IPv6 details without IPv6
structure	Unknown/missing top-level key, typo in a field, enum, required field, type, minimum, dangling and wrong-section references, invalid id, duplicates
rules	`runsOn` on non-virtual and on a wrong kind, `runsOn` + `inside`, `manages` outside a KVM and on itself, self reference, cycle, missing/non-inheritable/contradicting `site`, site without uplink, unused entries, inventory ≠ flake
unlock	No way to unlock, `threshold` too high or without methods, `nbde` without `tang` and the reverse, own Tang server, Tang loop, Tang server waiting for a human
hardware	`vendor`/`serial` on a VM, `model` on a VM allowed, serial twice; derived hardware fields and defaults
SSH keys	Wrong format, with comment, key data of another type, same key on two hosts; derived known_hosts entries and file
interfaces	MAC format, interface name, link-local value, MAC twice, multicast MAC, `eui64` from a private Wi-Fi MAC and without MAC, network at another site, `ipv4` without network, address per mode, reserved without MAC or DHCP server, address outside/broadcast/in pool/without prefix/twice; derived link-local (EUI-64 of a QEMU MAC, fixed, random) and VLAN
networks	Prefix format, VLAN range and duplicate per site (same id at another site allowed), host bits, pool outside/backwards/without server, DHCP server without interface or at another site; derived reservations

```
just inventory-test
```

builds `checks.<system>.inventory` (the real file is valid), `inventory-tests` (all cases pass), `inventory-schema` (schema fresh) and `inventory-sops` (host keys ↔ `.sops.yaml`). On failure the message lists each failing case with "want" and "got".

A new rule gets a case in the same commit: change one thing in `base` with `set`/`del`, paste the exact message. **Changed wording** of an error fails the tests – adjust case and this page together.

Workflows

What	Steps
New host / device	Add an entry to `inventory.json` (for a flake host `"managed": true` – otherwise `nix flake check` fails anyway), `just inventory-collect`, `network`/`ipv4` by hand, `just inventory-test`
Host key changed (reinstall)	`just inventory-collect --force <host>`, `.sops.yaml` + `sops updatekeys` (see Rotate keys), deploy, `just known-hosts`
New DHCP reservation	Interface with `mac`, `network`, `ipv4.mode: reserved` + `address`; the list for the router: `networks.<id>.reservations` in `just inventory`
Fill in a line's details	Fields in `uplinks.<id>`, nothing else to do
New field or kind	`spec` in `inventory.nix`, then `just inventory-schema`, a test case in `tests/inventory.nix`, this page (de/en)
Look at it	`just inventory`

Services

Services are named by function, not by product. There are three kinds of reachability: public directly, public behind the NetBird reverse proxy with SSO, or mesh only.

Name (`.homelab.example`)	App	Host	Reachable
`auth`	Pocket ID	zummi	public (nginx)
`mesh`	NetBird management/ dashboard	grammi	public (Traefik terminates TLS)
`s3.aldrin`	Garage S3 API	aldrin	public (nginx)
`rezepte`	Mealie	grammi	reverse proxy + SSO
`grist`	Grist	grammi	reverse proxy + SSO
`logs`	logchef	grammi	reverse proxy + SSO
`metrics`	Grafana	grammi	reverse proxy + SSO
`vm` / `vl`	VictoriaMetrics / VictoriaLogs UI	grammi	reverse proxy + SSO (via Traefik entrypoint `apps`, port 8090)
`s3-admin`	garage-ui	aldrin	mesh only
`dokumente`	Paperless + paperless-gpt (`/paperless-gpt/`)	ursalia	mesh only; from outside, through the reverse proxy, share links only behind a PIN (why)
`llm`	Ollama (behind Caddy)	zorlock	mesh only
`doku`	This documentation (private version)	aldrin + ursalia	in the mesh admin devices only (why), one A record per host; from outside a redirect to the public docs through the reverse proxy

Reverse proxy: Traefik on grammi owns port 443. It terminates `mesh.homelab.example` itself and hands everything else via TCP passthrough with PROXY protocol v2 to `netbird-proxy`, which does TLS and ACME (tls-alpn-01) on its own. Login at the proxy is SSO against Pocket ID, limited to the NetBird groups of the respective app; exception `dokumente` with a PIN.

Mesh-only names: Every name is its own NetBird DNS zone with one A record pointing at the mesh address (192.0.2.x). Public DNS does not know them; the certificate comes via DNS-01 through deSEC.

Across all Linux hosts: vmagent ships metrics, Vector ships journald and nginx logs to grammi. CrowdSec reads the nginx/Traefik logs on the public hosts. restic backups go to Garage buckets on aldrin, for Paperless additionally append-only to BorgBase.

Modules

The 17 modules under `modules/` carry the configuration shared by all hosts; every file explains in its header comment why it is the way it is. The public version contains them anonymised under `modules/` – as a reference, not directly importable: some read `hosts/<host>/*.yaml` and assume sops-nix. Four have options of their own.

Module	Purpose	Options / notes
auto-upgrade-git.nix	Nightly `nixos-rebuild switch`, commits and pushes the new `flake.lock` back to the repo	`services.autoUpgradeGit`: `enable`, `flakeDir`, `flakeAttr`, `updateInputs`, `dates`, `allowReboot`, `rebootWindow`, `deployKeySecret`, `gitAuthorEmail`
log-shipping.nix	journald and nginx logs to VictoriaLogs via Vector	`services.logShipping`: `enable`, `victoriaLogsUrl`
metrics-shipping.nix	vmagent scrapes the local exporters and remote-writes to VictoriaMetrics	`services.metricsShipping`: `enable`, `remoteWriteUrl`, `exporterPorts`
deploy.nix	The lab's policy for auto-upgrade-git: GitHub deploy key, pinned host key, reboot window 04:00–06:00	kept apart from the mechanism so that stays reusable
acme.nix	ACME defaults for every TLS host: DNS-01 against deSEC	token from `hosts/<host>/desec.yaml` (from Tofu)
netbird.nix	NetBird client in the mesh, registration via setup key, local metrics	setup key from `hosts/<host>/netbird.yaml` (from Tofu)
tailscale.nix	Shared Tailscale agent	`tailscale serve` stays in the host config
ssh.nix	Who may log in from where: root only via mesh or Tailscale	keys stay per host
firewall.nix	Only 22/80/443 public, tailnet as trusted interface	
crowdsec.nix	CrowdSec engine and firewall bouncer, whitelist for the lab hosts	workarounds for bugs in the nixpkgs modules, documented in the header
networking.nix	ifstate skeleton, systemd-resolved with DNS over TLS, no fallback resolvers	addresses and routes stay in the host config
metrics.nix	Prometheus exporters per host	protected by the firewall instead of bind addresses
logging.nix	Bounds journald, which otherwise grows up to 4 GB	

Module	Purpose	Options / notes
qemu-guest.nix	QEMU guest agent with an RPC deny list: snapshots yes, shell and file access over the channel no	
nix.nix	Nix daemon, garbage collection, store optimisation	
docs-site.nix	The private docs site (flake package `docs-site`) under `doku` on the mesh	`services.docsSite`: `enable`, `package`, `fqdn`, `allow` (default: the whole mesh; in the lab only the admin devices from `admin-peers.json`); on aldrin and ursalia
base.nix	Locale, console, time and tools for debugging	

just recipes

Recipe	Purpose
`just check <host>`	`git pull` + `nixos-rebuild dry-activate`
`just secrets <host>` / `pull-secrets <host>`	Edit host secrets on the host and fetch them back
`just topology`	Regenerate the topology dashboard (otherwise `nix flake check` fails)
`just inventory`	Show the validated inventory with everything derived (Inventory)
`just inventory-test`	Check `inventory.json`, run the validator tests, check the schema is fresh
`just inventory-schema`	Regenerate `inventory.schema.json` after a format change
`just inventory-collect [--force] [host …]`	Fetch SSH host keys, interfaces (MAC, medium, link-local) and hardware (vendor, model, product id, serial) from the devices into `inventory.json` – fills gaps and reports differences; `--force` takes the device values
`just known-hosts`	Write `~/.ssh/known_hosts.nixlab` on the MacBook from the inventory
`just netbird-plan` / `netbird-apply`	Change the NetBird account; apply also rewrites `expected-peers.json` (commit it, deploy grammi) and `admin-peers.json` (commit it, deploy aldrin and ursalia)
`just netbird-proxy-certs`	Get proxy services without a certificate one and bring the dashboard up to date (NetBird #7118); runs at the end of `netbird-apply`
`just dokumente-share-pin` / `dokumente-share-pin-show`	New random PIN for Paperless share links from outside (then `netbird-apply`), or show the current one
`just netbird-key <host>`	New setup key for a host, 24 h to deploy
`just netbird-import`	Restore lost Tofu state from the account
`just netbird-renew-token`	Yearly renewal of the service user token
`just github-key <host>`	New GitHub deploy key for a host (key via sops, registration via Tofu)
`just github-plan` / `github-apply`	Change the repositories and deploy keys on GitHub
`just desec-key <host>` / `desec-verify`	Rotate a deSEC token, or check the scopes from every host
`just gc <host>`	Delete store generations older than 7 days

Recipe	Purpose
`just docs`	Build this documentation: private PDFs (de/en) and the anonymised public version (README, PDFs, site sources, modules) in `public-docs/` (gitignored clone of `nixlab-docs`), committed and pushed there; GitHub builds the site from it
`just docs-serve`	Build the private site with real values locally and serve it at `http://127.0.0.1:8000`

Recipes that need sops on the MacBook run with a 1Password sign-in in front, which provides the age key via `SOPS_AGE_KEY_CMD`:

```
eval $(op signin) && export SOPS_AGE_KEY_CMD='op read "op://<vault>/<item>/password"' && just
        netbird-plan
```

Troubleshooting

- After a `nixos-rebuild`, `systemctl --failed` says nothing about the past – the switch clears failed states. Use `journalctl -u <unit>` instead.
- NetBird down, Tailscale still there: put `NIXLAB_DOMAIN=example-tailnet.ts.net` in front of the recipe.
- Tofu recipes refuse to run against an empty state, so no second `servers` group or set of tokens gets created.
- WLAN on ursalia: `iw dev wlp2s0f0 link` shows band, signal and PHY mode. The router's 5 GHz radio is on a DFS channel with a 300 TU beacon interval, so a single scan can miss it; band, signal, rates and PHY mode also go to VictoriaMetrics (node_exporter `wifi` collector plus `wifi_link_*`), Grafana dashboard "Homelab / WLAN".
- After a network change NetBird can keep a dead P2P path for minutes while still showing `Connected` (seen on ursalia, 2026-10-19). Fix: `systemctl restart netbird-mesh` on the host, via Tailscale if the mesh is down.
- A proxy service hangs at "Issuing certificate..." in the NetBird dashboard or fails the TLS handshake from outside ("internal error"): `just netbird-proxy-certs` – requests the names without a certificate from outside and restarts `netbird-proxy` on grammi (NetBird discussion #7118). Also runs at the end of `just netbird-apply`.

Conventions

- One git worktree per topic (`.worktrees/<topic>`), removed and its branch deleted when done. Tofu state always stays in the main checkout.
- Plan first, implement after approval.
- Subdomains by function (`rezepte`, not `mealie`).
- New web services on grammi use PostgreSQL instead of SQLite.
- Dual stack wherever possible.
- Code comments explain the *why*, including the mistakes that led to a decision, with a date.
- Code comments, commit messages and changelogs in English; the project documentation exists in German and English.
- The docs site loads nothing from third parties: the fonts and glightbox ship with the site (`docs/assets.nix`) instead of coming from Google Fonts or unpkg.com. Why: GDPR, and private docs that work without internet access.

Open items

☐ Move the NetBird mesh to dual stack (IPv6 overlay)
☐ Upstream PRs to the NetBird provider (#247 `direct_upstream`/cluster targets, #248 `crowdsec_mode`); then switch the proxy targets on grammi to `localhost`
☐ Remove Tailscale from the servers once nothing depends on it any more
☐ Grafana alerting (contact point still missing) for SMART, fwupd, Paperless, NetBird expected ≠ known, ursalia's WLAN (not associated, stuck on 2.4 GHz, weak signal)
☐ Update notifications for the inputs pinned by hand (NetBird, Pocket ID)
☐ Declarative root passwords via sops
☐ Keep the host list in one place instead of `flake.nix`, the Tofu modules and `justfile`